

Question 1

A data acquisition system continuously receives numerical readings from scientific instruments. At any point, the collected values are almost in ascending order, except for the most recently added elements. The system must reorder the data after each new entry with minimal additional memory usage.

Task:

1. Identify the most suitable sorting strategy for this situation.
2. **Write a C program** to implement this strategy.
3. Analyze the time complexity for best and worst cases.

Question 2

A big-data analytics engine processes extremely large datasets that cannot be loaded entirely into main memory. The system divides the data into smaller segments, sorts them independently, and then combines the results into a single sorted output.

Task:

1. Choose an appropriate sorting approach based on the scenario.
2. **Implement the solution in C**, clearly showing how the partial results are combined.
3. Explain how the approach supports scalability and external memory usage.

Question 3

A real-time transaction processing system needs to sort large arrays of integers frequently. The solution should be fast on average and should avoid using extra memory, as the system operates under strict space constraints. However, certain input patterns may degrade performance.

Task:

1. Determine the sorting technique that best fits these requirements.
2. **Develop a C program** implementing the algorithm.
3. Discuss the conditions under which performance degradation may occur.

Question 4

A national registry stores millions of fixed-length numeric identifiers. The sorting mechanism should avoid element-to-element comparisons and should preserve the relative order of identifiers with equal values.

Task:

1. Select the most appropriate sorting strategy for this dataset.
2. **Write a C program** to implement this method.
3. Justify why this approach is more efficient than comparison-based sorting in this context

Question 5

You are given the head of a linked list, which contains a series of integers separated by 0's. The beginning and end of the linked list will have `Node.val == 0`.

For every two consecutive 0's, merge all the nodes lying in between them into a single node whose value is the sum of all the merged nodes. The modified list should not contain any 0's.

Return the head of the modified linked list.

Example:



Input: head = [0,3,1,0,4,5,2,0]

Output: [4,11]

Explanation:

The above figure represents the given linked list.

The modified list contains

- The sum of the nodes marked in green: $3 + 1 = 4$.
- The sum of the nodes marked in red: $4 + 5 + 2 = 11$.